THE UNIVERSITY OF QUEENSLAND AUSTRALIA | CREATE CHANGE

ReLU Hull Approximation

Zhongkui Ma[1], Jiaying Li[2], Guangdong Bai[1]

[1]The University of Queensland

[2]Microsoft

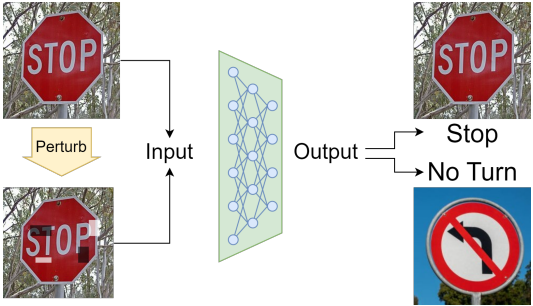
January, 2024
POPL'24, London

Background

Neural networks (NNs) have been applied to a variety of real-world applications due to their exceptional performance.

? ***Why need NN verification?***

Like programs, NNs are vulnerable to security threats and unexpected inputs. So, in addition to testing, NNs should be formally verified.



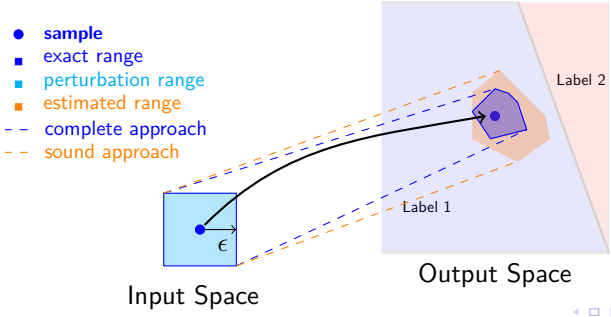
Neural Network Verification

What is NN verification?

To prove the correctness of neural networks, i.e., the outputs always satisfy a desired property with a set of specified inputs.

What to verify?

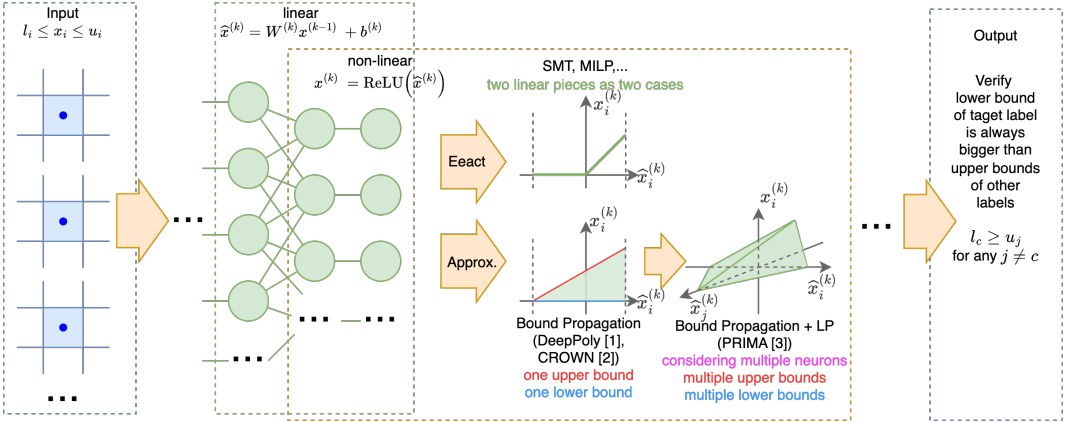
A typical property to verify is *local robustness*. For example, we want a sample with or without a tiny perturbation to be classified by the model as the same class.



Neural Network Verification

Difficulties non-enumerable perturbation & non-linearity of NN

Solution



[1] Gagandeep Singh, et al. An abstract domain for certifying neural networks. POPL 2019.
[2] Huan Zhang, et al. Efficient Neural Network Robustness Certification with General Activation Functions. NeurIPS 2018.
[3] Mark Niklas Müller, et al. 2022. PRIMA: general and precise neural network certification via scalable convex hull approximations. POPL 2022.

How to construct linear constraints ?

For linear constraints of activation function $y_i = \sigma(x_i)$, most approaches only consider constraints involved only x_i and y_i, called **single-neuron** linear constraints. So they construct a convex hull in (x_i, y_i)-space.

Constructing constraints involving multiple x_i and y_i, called **multiple-neuron** constraints, is hard and not efficient because of the computation of convex hulls/approximations.

Our work aims to construct the convex hull of the ReLU function.

WraLU - **Wra**pping the Re**LU** hull

Problem Statement

Input:

- an input polytope X in H-representation (constraints)

Output:

- a convex polytope M in H-representation (constraints) to over-approximate $(X, Y) := \{(\boldsymbol{x}, \boldsymbol{y}) \mid \boldsymbol{x} \in X, y_i = \mathrm{ReLU}(x_i), 1 \leq i \leq n\}$

Insight

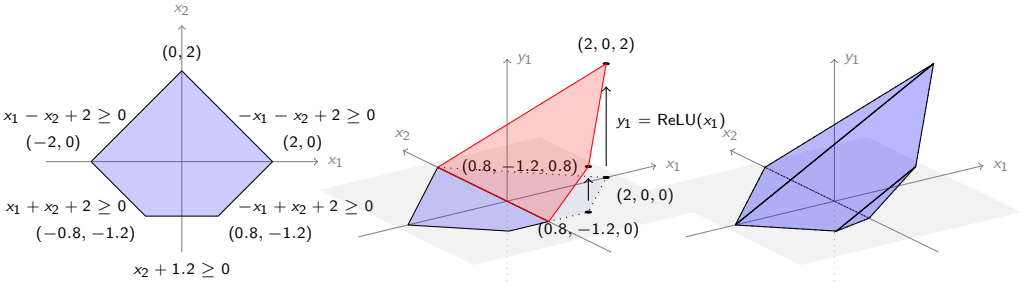
The ReLU function is a piece-wise linear function and two linear pieces determined by $y_i \geq 0$ and $y_i \geq x_i$ are taken as **lower faces** of the ReLU Hull.

These lower faces share edges, called **open edges**, with **upper faces**.

We construct the upper faces by *enumerating all combinations of open edges and vertices* of the lower faces.

Example

Given a polytope $X \subset (x_1, x_2)$-space with $y1 = \text{ReLU}(x1)$, $y2 = \text{ReLU}(x2)$. $(x1, x2, y1)$-space as an example, similar for y_2.



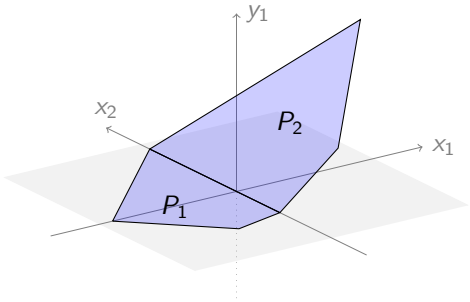
(a) X (b) (X, y_1) (c) ReLU Hull

Running Example

Obtain lower faces P_1 and P_2, and find upper faces

$$P_1 = X \cap \{(x_1, x_2, y_1) \mid y_1 = 0)\}, \qquad P_2 = X \cap \{(x_1, x_2, y_1) \mid y_1 = x_1)\}$$

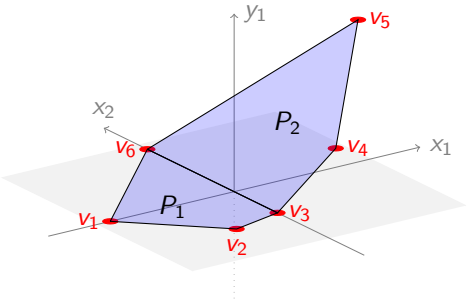
Based on P_1 and P_2, we first get two constraints $y_1 \geq 0$ and $y_1 \geq x_1$.



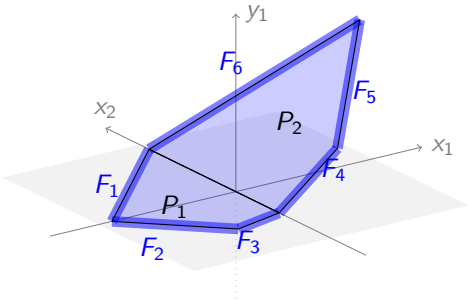
Identify vertices V and edges F of lower faces

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$F = \{F_1, F_2, F_3, F_4, F_5, F_6\}$$



(a) Vertices

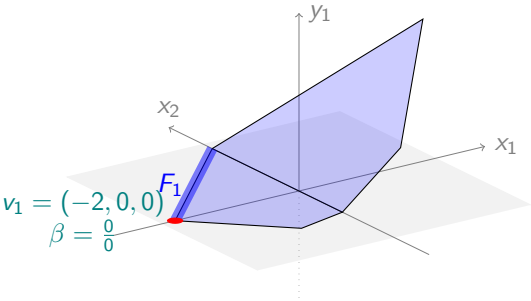


(b) Edges

⌛ Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



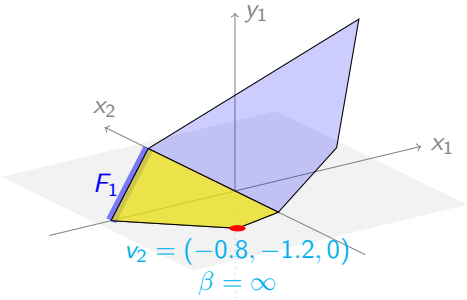
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$					
F_2						
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



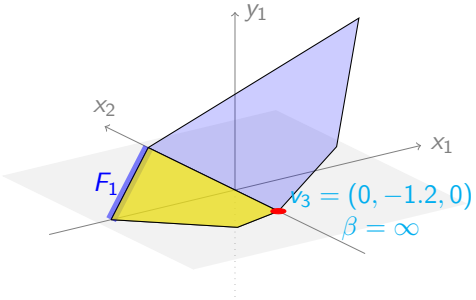
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞				
F_2						
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



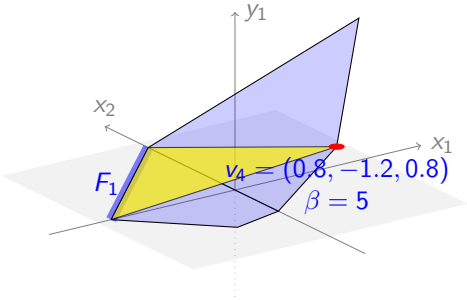
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞			
F_2						
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



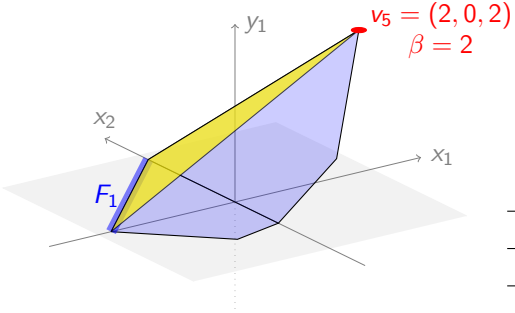
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5		
F_2						
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



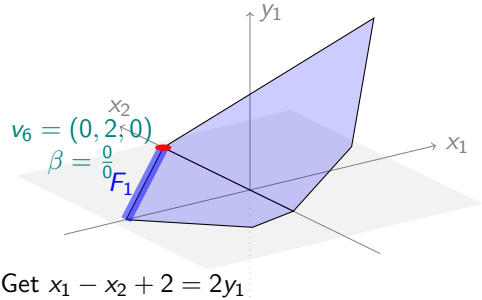
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	
F_2						
F_3						
F_4						
F_5						
F_6						

Constructing upper faces by (F_i, v_j)

$x_1 - x_2 + 2 = \beta y_1$ is a plane crossing F_1, $\beta = \frac{x_1 - x_2 + 2}{y_1}$

Substitute each vertex in this function to calculate β



- Get $x_1 - x_2 + 2 = 2y_1$

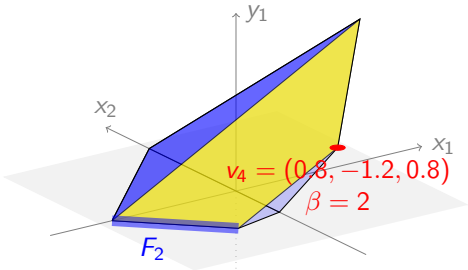
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2						
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_1 + x_2 + 2 = \beta y_1$ is a plane crossing F_2, $\beta = \frac{x_1+x_2+2}{y_1}$

Substitute each vertex in this function to calculate β



- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$

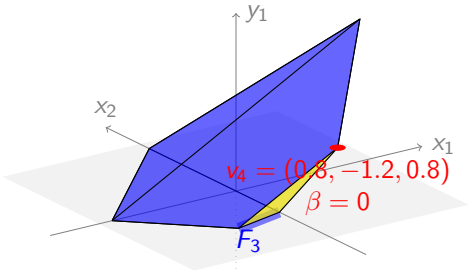
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2	$\frac{0}{0}$	$\frac{0}{0}$	∞	2	2	∞
F_3						
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_2 + 1.2 = \beta y_1$ is a plane crossing F_3, $\beta = \frac{x_1+1.2}{y_1}$

Substitute each vertex in this function to calculate β



- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$
- Get $x_2 + 1.2 = 0$

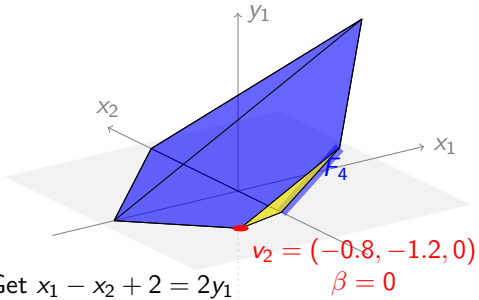
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2	$\frac{0}{0}$	$\frac{0}{0}$	∞	2	2	∞
F_3	∞	$\frac{0}{0}$	$\frac{0}{0}$	0	0.6	∞
F_4						
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$x_2 + 1.2 = \beta(y_1 - x_1)$ is a plane crossing F_4, $\beta = \frac{x_1+1.2}{y_1-x_1}$

Substitute each vertex in this function to calculate β



- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$
- Get $x_2 + 1.2 = 0$
- Get $x_2 + 1.2 = 0$

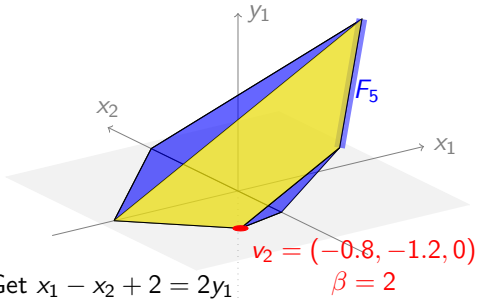
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2	$\frac{0}{0}$	$\frac{0}{0}$	∞	2	2	∞
F_3	∞	$\frac{0}{0}$	$\frac{0}{0}$	0	0.6	∞
F_4	0.6	0	$\frac{0}{0}$	$\frac{0}{0}$	∞	∞
F_5						
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$-x_1 + x_2 + 2 = \beta(y_1 - x_1)$ is a plane crossing F_5, $\beta = \frac{-x_1+x_2+2}{y_1-x_1}$

Substitute each vertex in this function to calculate β



- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$
- Get $x_2 + 1.2 = 0$
- Get $x_2 + 1.2 = 0$
- Get $-x_1 + x_2 + 2 = 2(y_1 - x_1)$

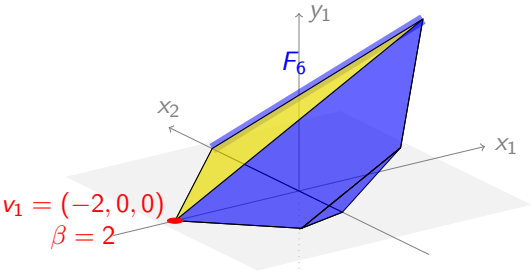
- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2	$\frac{0}{0}$	$\frac{0}{0}$	∞	2	2	∞
F_3	∞	$\frac{0}{0}$	$\frac{0}{0}$	0	0.6	∞
F_4	0.6	0	$\frac{0}{0}$	$\frac{0}{0}$	∞	∞
F_5	2	2	∞	$\frac{0}{0}$	$\frac{0}{0}$	∞
F_6						

⌛ Constructing upper faces by (F_i, v_j)

$-x_1 - x_2 + 2 = \beta(y_1 - x_1)$ is a plane crossing F_6, $\beta = \frac{-x_1 - x_2 + 2}{y_1 - x_1}$

Substitute each vertex in this function to calculate β



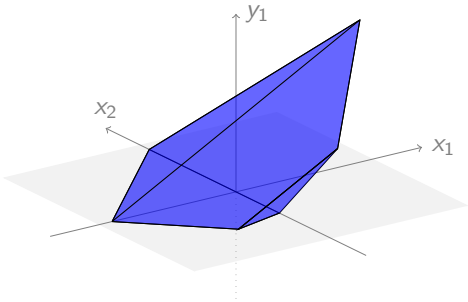
- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$
- Get $x_2 + 1.2 = 0$
- Get $x_2 + 1.2 = 0$
- Get $-x_1 + x_2 + 2 = 2(y_1 - x_1)$
- Get $-x_1 - x_2 + 2 = 2(y_1 - x_1)$

- $(\beta = \frac{0}{0})$ vertex on edge
- $(\beta = \infty)$ a plane containing a lower face
- (not minimal β) a plane splitting a lower face
- (minimal β) a plane determining an upper face

	v_1	v_2	v_3	v_4	v_5	v_6
F_1	$\frac{0}{0}$	∞	∞	5	2	$\frac{0}{0}$
F_2	$\frac{0}{0}$	$\frac{0}{0}$	∞	2	2	∞
F_3	∞	$\frac{0}{0}$	$\frac{0}{0}$	0	0.6	∞
F_4	0.6	0	$\frac{0}{0}$	$\frac{0}{0}$	∞	∞
F_5	2	2	∞	$\frac{0}{0}$	$\frac{0}{0}$	∞
F_6	2	5	∞	∞	$\frac{0}{0}$	$\frac{0}{0}$

Running Example

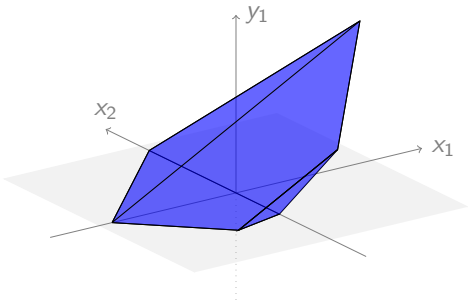
Constructing upper faces by (F_i, v_j)



We get three planes after removing repeated ones.

- Get $x_1 - x_2 + 2 = 2y_1$
- Get $x_1 + x_2 + 2 = 2y_1$
- Get $x_2 + 1.2 = 0$
- ~~Get $x_2 + 1.2 = 0$~~
- ~~Get $-x_1 + x_2 + 2 = 2(y_1 - x_1)$~~
- ~~Get $-x_1 - x_2 + 2 = 2(y_1 - x_1)$~~

Constructing upper faces by (F_i, v_j)



Convert them to linear constraints by ensuring the origin satisfies them.

- $x_1 - x_2 + 2 \geq 2y_1$
- $x_1 + x_2 + 2 \geq 2y_1$
- $x_2 + 1.2 \geq 0$

Additionally, two lower faces determine two constraints.

- $y_1 \geq 0$
- $y_1 \geq x_1$

General high-dimensional cases

- Lower faces $\Rightarrow$ $y_i \geq 0$ and $y_i \geq x_i$
- Upper faces $\Rightarrow$ $\boldsymbol{ax} + b \geq \beta y_i$ or $\boldsymbol{ax} + b \geq \beta(y_i - x_i)$, where $\boldsymbol{ax} + b$ is from constraints of X
  - The minimal β determined by a vertex gives an upper-face constraint.

Innovation 1 - Extend one output dimension y_i at one time.

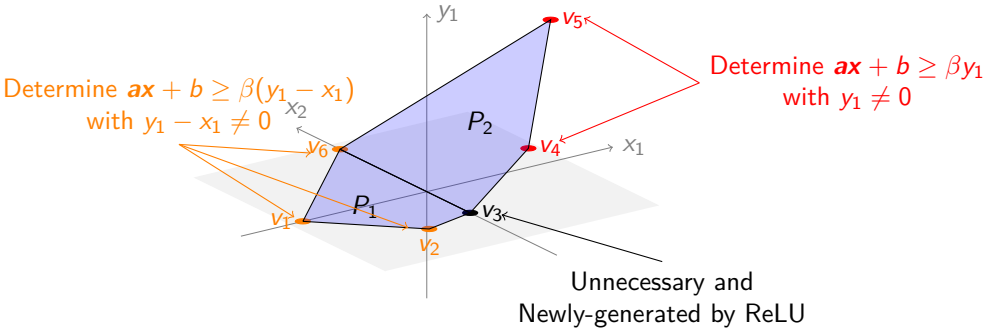
We calculate one dimension y_i at one time, in which we locate the upper face by one vertex in the new dimension.

$$
\begin{aligned}
\boldsymbol{ax} + b \geq \beta_{11} y_1 \quad &\text{or} \quad & \boldsymbol{ax} + b \geq \beta_{12}(y_1 - x_1) \\
\boldsymbol{ax} + b + \beta_{11} y_1 + \beta_{12}(y_1 - x_1) \geq \beta_{21} y_2 \quad &\text{or} \quad & \boldsymbol{ax} + b + \beta_{11} y_1 + \beta_{12}(y_1 - x_1) \geq \beta_{22}(y_2 - x_2) \\
& \cdots & \\
\boldsymbol{ax} + b + \textstyle\sum_{i=1}^{n-1} \beta_{i1} y_i + \beta_{i2}(y_i - x_i) \geq \beta_{n1} y_n \quad &\text{or} \quad & \boldsymbol{ax} + b + \textstyle\sum_{i=1}^{n-1} \beta_{i1} y_i + \beta_{i2}(y_i - x_i) \geq \beta_{n2}(y_n - x_n)
\end{aligned}
$$

Because of this successive calculation, our approach is ***incomplete***.

Innovation 2 - Stable number of considered vertices.

We can filter vertices by satisfying $y_i \neq 0$ or $y_i \neq x_i$ to determine upper-face constraints. We only consider those vertices transformed from X, which are necessary for determining upper-face constraints. Because other vertices have the same β value as vertices of X.



***Innovation 3* - Stable number of considered edges.**

We simultaneously calculate $\boldsymbol{ax} + b \geq \beta_1 y_i$ and $\boldsymbol{ax} + b \geq \beta_2(y_i - x_i)$ for lazy selection of edges. Choose the minimal values of β_1 and β_2 to get a valid constraint, and then choose the bigger one of both to exclude the non-existing edge. So the number of upper-face constraints is always the same as the number of constraints of X.

Constraints of X:

- $x_1 - x_2 + 2 \geq 0$
- $x_1 + x_2 + 2 \geq 0$
- $x_2 + 1.2 \geq 0$
- $-x_1 + x_2 + 2 \geq 0$
- $-x_1 - x_2 + 2 \geq 0$

Upper-faced constraints of (X, y_1) excluding lower-face constraints:

- $x_1 - x_2 + 2 \geq 2y_1$
- $x_1 + x_2 + 2 \geq 2y_1$
- $x_2 + 1.2 \geq 0$
- $-x_1 + x_2 + 2 \geq 2(y_1 - x_1)$
- $-x_1 - x_2 + 2 \geq 2(y_1 - x_1)$

Evaluation

ReLU Hull Approximation

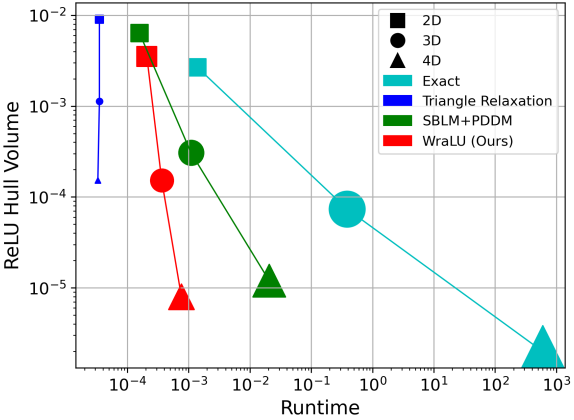
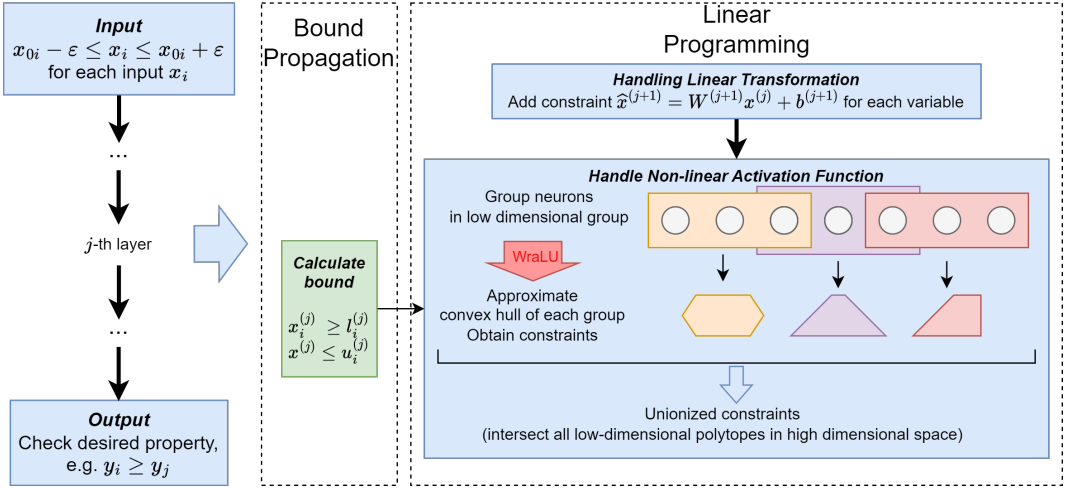


Figure: Performance of exact/approximate algorithms for ReLU hull in different dimensions (The marker size represents the number of constraints (log scale).)

Evaluation

Implementation



Evaluation

ReLU Neural Network Verification

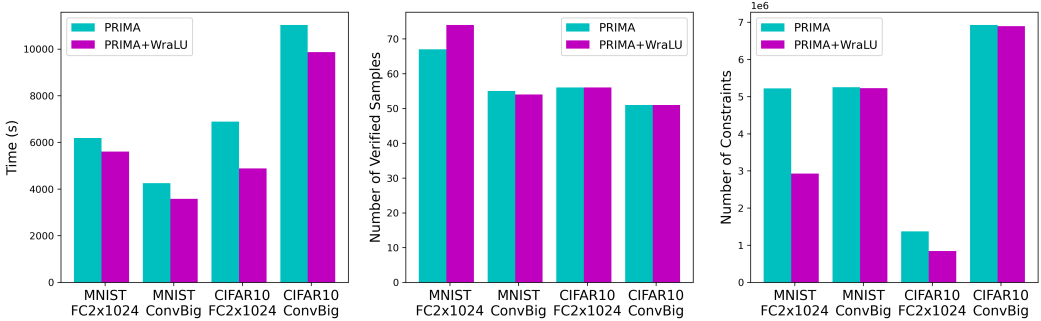


Figure: Performance of different approaches on neural network verification (100 correctly classified samples are chosen as candidates.)

Evaluation

ReLU Neural Network Verification

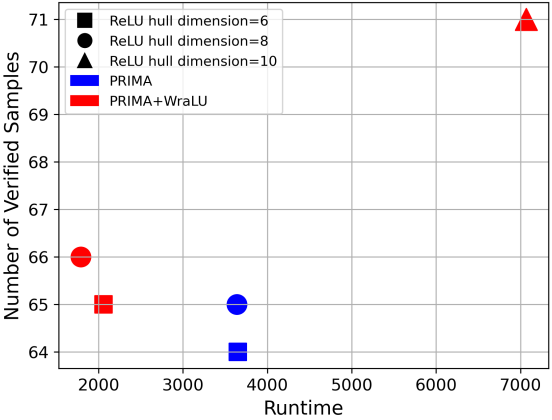


Figure: Performance of different ReLU hull dimensions on neural network verification (100 correctly classified samples are chosen as candidates.This network is a fully-connected network with 3 hidden layers and 50 neurons in each hidden layer.)

Future Work

- generalization to **convex and piece-wise linear functions**, e.g., LeakyReLU, MaxPool.
  All linear pieces are considered to be lower faces and new faces are created using the same techniques as WraLU.

- generalization to **piece-wise linear functions**, e.g., ELU, and **non-piece-wise differentiable functions**, e.g., Tanh and Sigmoid.
  Two piece-wise linear functions are constructed as the lower and upper bounds of the function. Then the lower and upper faces can be constructed by these linear-wised functions.

Thank you

zhongkui.ma@uq.edu.au

Source code is available at https://github.com/UQ-Trust-Lab/WraLU